LAPORAN
PROYEK AKHIR MOBILE COMPUTING
SISTEM MONITORING SUHU DAN KELEMBABAN BERBASIS INTERNET OF THINGS (IOT) MENGGUNAKAN MIKROKONTROLER ESP32

Disusun Oleh Kelompok 2:

Nur Nabila Amalia (2407412015)

Zulfarida (2421010231)

Namira Assalwa (2407412025)

Muhammad Adib Arkan (2407412018)

JURUSAN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK NEGERI JAKARTA

2026

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Pemantauan dan pengendalian parameter fisik lingkungan seperti suhu (*temperature*) dan kelembaban relatif (*relative humidity*) memiliki signifikansi yang sangat tinggi di berbagai sektor industri modern. Beberapa contoh nyata penerapannya meliputi ruang pusat data (*data center* atau *server room*), gudang logistik farmasi, industri pengolahan makanan, hingga sistem otomasi hunian pintar (*smart home*). Pada fasilitas-fasilitas tersebut, fluktuasi iklim mikro yang tidak terkendali dapat berakibat fatal. Suhu yang terlalu tinggi dapat memicu kegagalan sistem akibat panas berlebih (*thermal runaway*) pada perangkat keras elektronik. Sementara itu, tingkat kelembaban yang terlalu tinggi dapat mengakibatkan kondensasi uap air yang memicu korosi dan hubungan arus pendek (*short circuit*), sedangkan kelembaban yang terlalu rendah dapat menumpuk listrik statis yang merusak komponen semikonduktor sensitif.

Secara konvensional, pemantauan parameter lingkungan ini dilakukan secara manual menggunakan termometer atau higrometer analog. Metode ini memiliki banyak kelemahan, antara lain tingginya potensi kesalahan manusia (*human error*), ketiadaan pencatatan riwayat data secara kontinu, serta ketidakmampuan memberikan respons cepat (mitigasi) saat terjadi anomali kondisi. Guna menjawab tantangan tersebut, teknologi *Internet of Things* (IoT) hadir sebagai solusi terintegrasi. IoT memungkinkan perangkat keras penginderaan mengumpulkan data secara berkala dan mentransmisikannya secara otomatis ke platform digital melalui jaringan internet maupun intranet.

Dalam praktikum ini, dikembangkan sebuah sistem monitoring suhu dan kelembaban berbasis IoT dengan menggunakan mikrokontroler ESP32 dan sensor DHT22. Sistem ini dirancang untuk mendeteksi metrik iklim secara *real-time*, memproses data secara lokal untuk menentukan status kedaruratan lingkungan (NORMAL, PERINGATAN, BAHAYA), memberikan peringatan audiovisual melalui kombinasi lampu indikator LED (Tri-LED) dan *Buzzer* aktif, serta menayangkan data telemetri tersebut pada sebuah *Web Dashboard* interaktif yang dibangun menggunakan pustaka Streamlit di sisi server pengawas. Jalur komunikasi

data dirancang menggunakan protokol jaringan ringan *Message Queuing Telemetry Transport* (MQTT) melalui *broker* Eclipse Mosquitto lokal demi menjamin latensi yang sangat rendah dan keandalan transmisi tanpa ketergantungan pada koneksi internet publik.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam pelaksanaan praktikum ini adalah sebagai berikut:

1. Bagaimana mengintegrasikan dan merangkai perangkat keras (*hardware node*) berbasis mikrokontroler ESP32 dengan sensor DHT22, layar LCD 16x2 I2C, serta komponen aktuator lokal (LED merah, LED hijau, dan *Buzzer*)?
2. Bagaimana merancang dan menerapkan logika pembagian kondisi lingkungan (*state machine/triage*) ke dalam *firmware* ESP32 untuk mengendalikan aktuator secara otonom?
3. Bagaimana mengonfigurasi jalur komunikasi data nirkabel menggunakan protokol MQTT dengan memanfaatkan *broker* lokal (*localhost*) untuk mencapai transmisi data berlatensi rendah?
4. Bagaimana merancang skrip visualisasi pada sisi server menggunakan Python dan Streamlit agar mampu menampilkan grafik tren dan tabel log riwayat sensor secara *real-time* tanpa memicu kendala kegagalan *threading*?

1.3 Tujuan Praktikum

Tujuan yang ingin dicapai melalui pelaksanaan praktikum ini meliputi:

1. Memahami arsitektur perangkat keras dan pinout mikrokontroler ESP32 serta mekanisme akuisisi data digital dari sensor DHT22.
2. Mampu membuat logika pemrosesan bersyarat (*triage logic)* pada firmware mikrokontroler untuk mengaktifkan peringatan dini berupa indikator LED dan alarm Buzzer.
3. Mampu melakukan instalasi, konfigurasi, dan pengujian transmisi data nirkabel menggunakan protokol MQTT melalui perantara broker Eclipse Mosquitto lokal.
4. Mampu membangun antarmuka web (*Web Dashboard*) interaktif berbasis Streamlit yang mengadopsi mekanisme synchronous polling MQTT untuk menampilkan grafik fluktuasi iklim mikro secara real-time dan stabil.

1.4 Manfaat Praktikum

Praktikum ini memberikan manfaat dalam pemahaman konsep rancang bangun sistem IoT ujung-ke-ujung (*end-to-end*), mulai dari lapisan fisik (*physical layer*) berupa sensor dan mikrokontroler, lapisan jaringan (*network layer*) menggunakan MQTT, hingga lapisan aplikasi (*application layer*) berupa web dasbor visualisasi. Selain itu, praktikum ini melatih kemampuan analisis pemecahan masalah (*troubleshooting*) terkait keterbatasan catu daya pin GPIO pada aktuator induktif/audio serta penanganan perselisihan komputasi (*threading conflict*) pada *framework* aplikasi web reaktif.

BAB 2
METODE

2.1 Mikrokontroler ESP32 NodeMCU

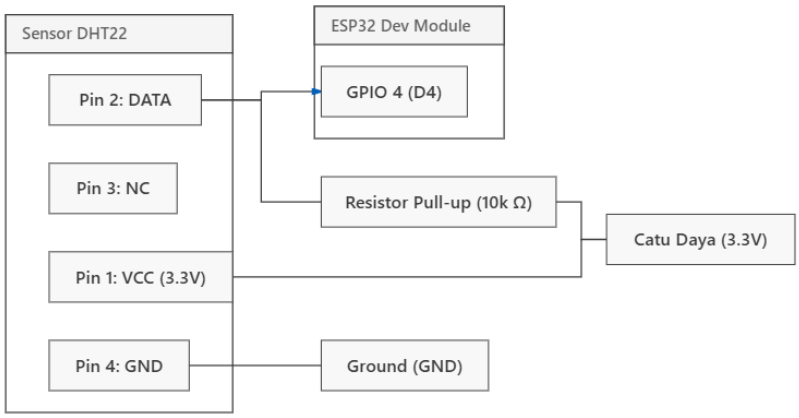
ESP32 adalah mikrokontroler *System-on-a-Chip* (SoC) berbiaya rendah dan berdaya rendah yang dikembangkan oleh Espressif Systems. Chip ini dilengkapi dengan fitur konektivitas nirkabel Wi-Fi (802.11 b/g/n) dan Bluetooth (v4.2 BR/EDR dan BLE) yang terintegrasi secara langsung di dalamnya. Mikrokontroler ini ditenagai oleh prosesor Tensilica Xtensa Dual-Core 32-bit LX6 yang berjalan pada frekuensi clock hingga 240 MHz. ESP32 memiliki memori internal berupa SRAM sebesar 520 KB dan mendukung memori flash eksternal hingga 16 MB.

Berikut adalah beberapa fitur unggulan ESP32 yang mendasari penggunaannya dalam proyek IoT:

1. **Konektivitas Nirkabel**: Wi-Fi internal memudahkan integrasi perangkat dengan jaringan lokal maupun internet tanpa membutuhkan modul tambahan seperti ESP8266 eksternal.
2. **GPIO yang Fleksibel**: Memiliki puluhan pin GPIO (*General Purpose Input/Output*) yang mendukung fungsi ADC (*Analog-to-Digital Converter*), DAC (*Digital-to-Analog Converter*), PWM (*Pulse Width Modulation*), I2C, SPI, dan UART.
3. **Daya Rendah**: Mendukung berbagai mode hemat daya (*sleep modes*) untuk aplikasi sensor berbasis baterai.

2.2 Sensor Suhu dan Kelembaban DHT22 (AM2302)

Sensor DHT22 (dikenal juga dengan nama komersial AM2302) adalah sensor digital termal dan kelembaban relatif yang memiliki performa tinggi dengan harga ekonomis. Sensor ini menggunakan transduser kelembaban kapasitif (capacitive humidity sensor) untuk mendeteksi jumlah uap air di udara dan sebuah termistor NTC (Negative Temperature Coefficient) untuk mengukur suhu udara sekitar.



Sensor DHT22 menggunakan protokol komunikasi satu kabel (*single-wire bus*) khusus yang mentransmisikan sinyal digital terkalibrasi ke mikrokontroler. Proses komunikasi diawali dengan ESP32 mengirimkan sinyal pemicu (*start signal*), yang kemudian direspons oleh sensor dengan mengirimkan data paket digital sebanyak 40 bit. Paket data tersebut terdiri dari:

1. 16 bit data kelembaban relatif (skala integer dikalikan 10).
2. 16 bit data suhu (skala integer dikalikan 10, bit tertinggi menentukan tanda positif/negatif).
3. 8 bit data *checksum* untuk validasi data dari kesalahan transmisi.

Spesifikasi teknis sensor DHT22 meliputi:

1. Akurasi suhu: $\pm 0.5^{\circ}$ dengan resolusi $0.1^{\circ}C$ (jangkauan $-40^{\circ}C$ hingga $80^{\circ}C$).
2. Akurasi kelembaban: $\pm 2\%$ R dengan resolusi 0.1% RH (jangkauan 0% hingga 100% RH).
3. Waktu pengambilan sampel (*sampling rate*): minimal 2 detik sekali.

2.3 Layar LCD 16x2 dengan Modul Driver I2C (PCF8574)

Layar LCD *(Liquid Crystal Display)* 16x2 digunakan untuk menampilkan data suhu, kelembaban, dan status secara lokal pada perangkat keras. Penggunaan LCD karakter standar membutuhkan minimal 6 hingga 10 pin data mikrokontroler (mode 4-bit atau 8-bit), yang sangat memboroskan sumber daya GPIO ESP32.

Untuk mengatasi keterbatasan ini, digunakan modul ekspansi I2C berbasis IC PCF8574. Modul ini bertindak sebagai konverter data paralel ke serial I2C. Melalui protokol I2C (Inter-Integrated Circuit), mikrokontroler hanya memerlukan 2 pin komunikasi yaitu SDA (Serial Data) dan SCL (Serial Clock) untuk mengontrol LCD

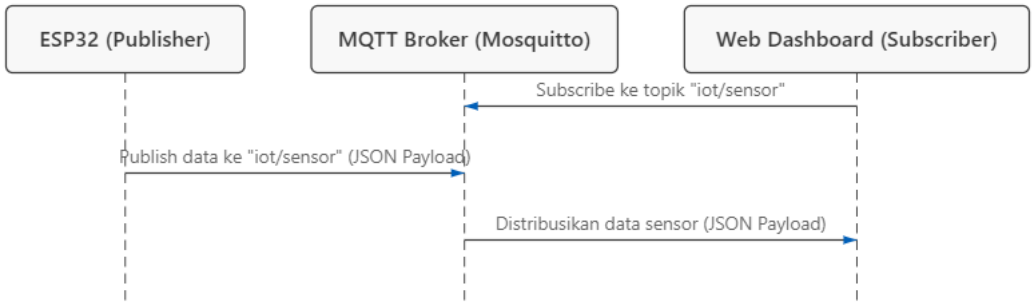
secara penuh. IC PCF8574 memiliki alamat perangkat keras default (biasanya `0x27` atau `0x3F`) yang dapat dipindai melalui program I2C Scanner untuk mendeteksi keberadaannya di dalam bus data komunikasi. Dalam implementasi praktis proyek ini, modul LCD I2C berhasil diintegrasikan secara optimal pada alamat `0x27` untuk menyajikan data telemetri secara langsung di lokasi fisik perangkat keras.

2.4 Protokol Komunikasi MQTT dan Broker Eclipse Mosquitto

Message Queuing Telemetry Transport (MQTT) adalah protokol komunikasi berbasis standar ISO (ISO/IEC PRF 20922) yang berjalan di atas protokol TCP/IP. MQTT didesain khusus sebagai protokol pengiriman pesan bertopologi *Publish-Subscribe* yang sangat ringan (*lightweight*), asinkron, dan membutuhkan pita lebar (*bandwidth*) minimal. Protokol ini sangat ideal untuk perangkat IoT dengan keterbatasan daya pemrosesan, memori, dan kualitas koneksi jaringan.

Komponen utama arsitektur MQTT terdiri atas:

1. **Publisher (Penerbit):** Klien yang mengirimkan pesan telemetri pada topik tertentu. Dalam proyek ini, ESP32 bertindak sebagai Publisher.
2. **Subscriber (Pelanggan):** Klien yang mendaftarkan diri ke topik tertentu untuk menerima data secara otomatis setiap kali ada pembaruan dari Publisher. Dalam proyek ini, Web Dashboard Streamlit bertindak sebagai Subscriber.
3. **Broker (Perantara):** Server pusat yang bertugas menerima semua pesan masuk dari Publisher, menyaring pesan berdasarkan topik, dan mendistribusikannya ke seluruh Subscriber yang terdaftar. Eclipse Mosquitto digunakan sebagai broker lokal dalam rancangan ini.
4. **Topic (Topik):** Kunci string terstruktur (misalnya `iot/sensor`) yang digunakan oleh broker untuk mengklasifikasikan pesan.



2.5 Web Framework Python Streamlit

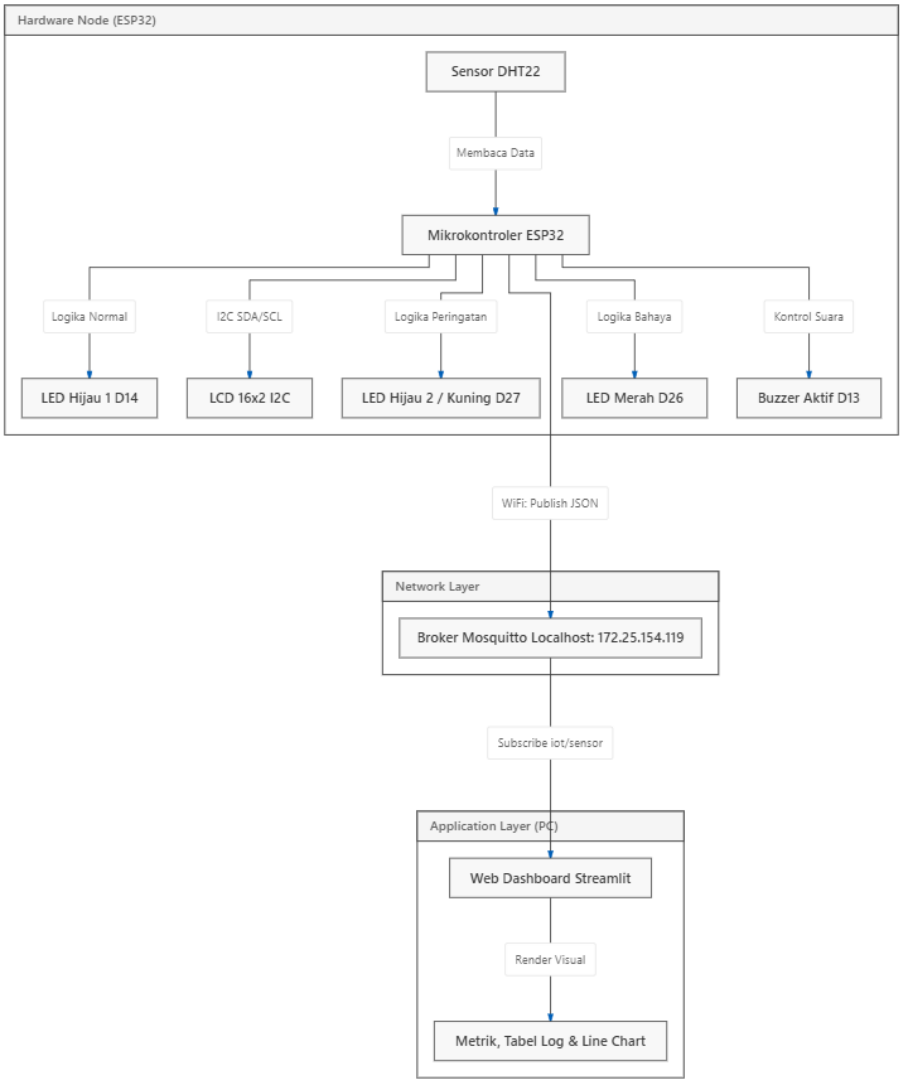
Streamlit adalah kerangka kerja (framework) berbasis Python sumber terbuka yang dirancang khusus untuk membangun aplikasi web interaktif untuk visualisasi data dan demonstrasi model kecerdasan buatan dengan cepat. Dibandingkan dengan pengembangan web tradisional yang memerlukan penguasaan HTML, CSS, JavaScript, serta penanganan backend (seperti Flask/Django), Streamlit memungkinkan pembuatan antarmuka web yang kaya fitur hanya dengan menuliskan kode Python murni.

Streamlit beroperasi menggunakan paradigma eksekusi linier: setiap kali terjadi interaksi pengguna pada antarmuka (seperti penekanan tombol) atau dipicu pemanggilan penyegaran manual `(st.rerun())`, seluruh skrip Python akan dieksekusi ulang dari baris pertama hingga terakhir. Guna mempertahankan nilai variabel agar tidak terhapus saat siklus eksekusi ulang berlangsung, Streamlit menyediakan mekanisme penyimpanan persisten dalam memori yang dinamakan `st.session_state`.

BAB 3
HASIL DAN DISKUSI

3.1 Diagram Blok Sistem

Arsitektur interkoneksi komponen dalam sistem monitoring ini dibagi menjadi tiga lapisan utama: Lapisan Sensor/Aktuator (Fisik), Lapisan Komunikasi (Broker MQTT), dan Lapisan Visualisasi (Dasbor).



3.2 Skema Rangkaian Hardware dan Pin Mapping

Rangkaian perangkat keras disusun menggunakan papan percobaan (*breadboard*) dengan skema kelistrikan yang mengaitkan GPIO ESP32 ke pin-pin sensor dan aktuator. Berikut adalah tabel pemetaan pin (*pin mapping*) dari komponen-komponen tersebut:

Nama Komponen	Pin Perangkat	Pin GPIO ESP32	Warna Kabel	Keterangan / Fungsi
Sensor DHT22	VCC	3.3V	Merah	Catu daya sensor (3.3 Volt DC)
	DATA	GPIO 4 (D4)	Biru	Jalur data digital (*single-wire)*
	GND	GND	Hitam	Grounding bersama
LCD 16x2 I2C	VCC	Vin (5V)	Merah	Catu daya LCD (5 Volt DC untuk tingkat kecerahan karakter yang optimal)
	GND	GND	Hitam	Grounding bersama
	SDA	GPIO 21	Kuning	Jalur data komunikasi I2C
	SCL	GPIO 22	Putih	Jalur clock komunikasi I2C
LED Hijau 1	Anoda (+)	GPIO 33 (D33)	Ungu	Indikator kondisi NORMAL
(Kondisi Normal)	Katoda (-)	GND (via resistor 220Ω)	Hitam	Proteksi kelebihan arus
LED Hijau 2	Anoda (+)	GPIO 27 (D27)	Hijau	Indikator kondisi PERINGATAN
(Kondisi Waspada)	Katoda (-)	GND (via resistor 220Ω)	(Kondisi Waspada)	Katoda (-)
LED Merah	Anoda (+)	GPIO 26 (D26)	Abu-abu/Hitam	Indikator kondisi BAHAYA
(Kondisi Bahaya)	Katoda (-)	GND (via resistor 220Ω	(Kondisi Bahaya)	Katoda (-)
Buzzer Aktif	Positif (+)	GPIO 33 (D33)	Hitam	Alarm audio (Sistem membutuhkan tipe 3V, namun menggunakan komponen 5V yang ada)
	Negatif (-)	GND	Hitam	Grounding bersama

3.3 Desain Alur Logika (State Machine) & Algoritma Triage

Sistem mengadopsi logika bersyarat berbasis parameter kenyamanan lingkungan fisis. Ambang batas teoretis yang didefinisikan dalam *firmware* adalah:

- Ambang Batas Suhu Ideal (T): $25.0°\text{C} \leq T \leq 35.0°\text{C}$
- Ambang Batas Kelembaban Ideal (H): $60.0\%\ \text{RH} \leq H \leq 80.0\%\ \text{RH}$

Berdasarkan nilai tersebut, dirancang mesin logika dengan tiga kemungkinan status:

1. **NORMAL:** Kondisi di mana nilai suhu (T) DAN kelembaban (H) berada dalam batas ideal. Status = NORMAL $(25.0 \leq T \leq 35.0) \wedge (60.0 \leq H \leq 80.0)$ *Aksi aktuator*: LED Hijau 1 menyala (HIGH), LED Hijau 2 mati (LOW), LED Merah mati (LOW), Buzzer mati (LOW).
2. **BAHAYA:** Kondisi di mana nilai suhu (T) DAN kelembaban (H) secara bersamaan berada di luar batas ideal (suhu tidak ideal dan kelembaban tidak ideal). Status = BAHAYA $\neg(25.0 \leq T \leq 35.0) \wedge \neg(60.0 \leq H \leq 80.0)$ *Aksi aktuator*: LED Hijau 1 mati (LOW), LED Hijau 2 mati (LOW), LED Merah menyala (HIGH), Buzzer menyala secara kontinu (HIGH).
3. **PERINGATAN:** Kondisi transisi di mana hanya salah satu parameter saja yang tidak ideal (baik hanya suhu yang tidak ideal, atau hanya kelembaban saja yang tidak ideal). Status = PERINGATAN$[\neg(25.0 \leq T \leq 35.0) \oplus \neg(60.0 \leq H \leq 80.0)]$ *Aksi aktuator*: LED Hijau 1 mati (LOW), LED Hijau 2 menyala (HIGH), LED Merah mati (LOW), Buzzer menyala secara kontinu (HIGH).

Pemrosesan logika ini dijalankan secara berulang dalam fungsi `loop()` firmware ESP32 dengan siklus pembacaan data setiap 5.000 milidetik (5 detik).

3.4 Desain Arsitektur Jaringan dan Paket Data MQTT

Sistem komunikasi mengandalkan transmisi asinkron menggunakan protokol MQTT. Broker dijalankan secara lokal di PC pengawas menggunakan Eclipse Mosquitto dengan alamat IP intranet `172.25.154.119` pada port default `1883`. Pilihan ini memiliki keunggulan latensi transmisi yang sangat minim (kurang dari 10 milidetik) dibandingkan menggunakan broker cloud publik (seperti `broker.hivemq.com`) yang memerlukan koneksi gateway internet publik dengan latensi tak menentu.

Data dari sensor dikemas ke dalam payload berbentuk format obyek JSON *(JavaScript Object Notation)* yang ringkas untuk menghemat bandwidth:



Untuk mengolah JSON secara efisien di mikrokontroler tanpa memicu fragmentasi memori RAM *(heap memory fragmentation)*, digunakan pustaka `ArduinoJson` berbasis alokasi memori statis sebesar 128 byte menggunakan obyek `StaticJsonDocument<128>`.

3.5 Desain Visualisasi Web Dashboard Streamlit

Antarmuka web pengawas dirancang untuk menampilkan data telemetri secara berkelanjutan *(real-time* data *log stream).* Elemen antarmuka terdiri dari:

1. **Metrik Instan:** Menampilkan angka digital besar untuk metrik Suhu (°C) dan Kelembaban (%).
2. **Status Banner:** Komponen visual dinamis yang berubah warna berdasarkan pesan status yang diterima (hijau 1 untuk NORMAL, hijau 2 untuk PERINGATAN, dan merah untuk BAHAYA).
3. **Tabel Runtunan Data (Log):** Menyajikan daftar riwayat data telemetri yang diurutkan secara terbalik (reverse chronological order), di mana baris data terbaru selalu berada di paling atas. Kapasitas tabel dibatasi maksimal 30 baris untuk mencegah penurunan performa browser akibat kelebihan beban memori DOM.
4. **Grafik Fluktuasi Iklim (Line Chart):** Grafik garis dinamis yang memetakan tren suhu dan kelembaban relatif sepanjang waktu.

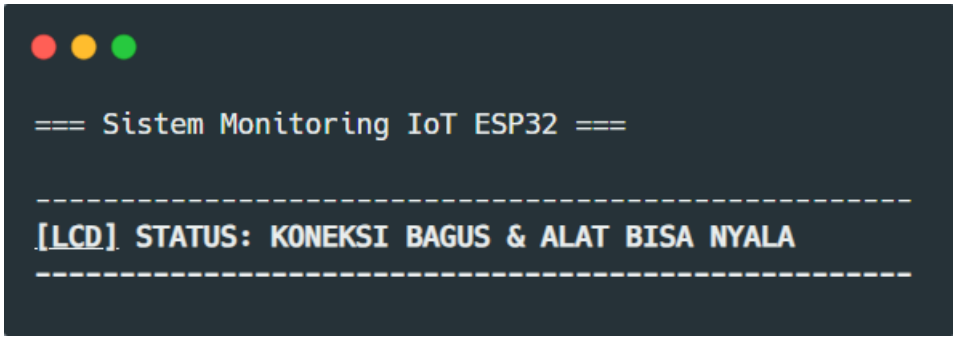
Aplikasi web dirancang menggunakan skema *Synchronous Polling* MQTT agar selaras dengan mekanisme pemuatan ulang Streamlit secara deterministik, tanpa memicu interferensi dari daemon thread eksternal yang umumnya tidak kompatibel dengan visualisasi runtime Streamlit.

BAB 4
HASIL DAN PEMBAHASAN

4.1 Implementasi Hardware Node Sensor dan Pengujian I2C LCD

Rangkaian fisik perangkat keras disusun dan dipasang pada papan proyek. ESP32 berhasil terhubung ke jaringan WiFi lokal "ㅎㅎㅎㅎㅎ" dengan penyediaan alamat IP dinamis DHCP. Setelah terkoneksi ke jaringan intranet lokal, dilakukan pengujian terhadap bus komunikasi I2C LCD menggunakan kode I2C Scanner bawaan.

Hasil pengujian fisik menunjukkan bahwa layar LCD 16x2 I2C berfungsi secara optimal tanpa kendala. Program scanner I2C pada Serial Monitor mendeteksi keberadaan chip PCF8574 pada alamat hardware `0x27` dengan log keluaran sebagai berikut:



Setelah dilakukan pengunggahan program dengan alamat inisialisasi `0x27`, layar LCD berhasil terinisialisasi secara sempurna, ditandai dengan menyalanya lampu latar (backlight) secara terang dan munculnya karakter teks metrik lingkungan. Pada baris pertama LCD, sistem menayangkan data suhu dan kelembaban waktu-nyata (misalnya, `T:27.2C H:67.3%`), sedangkan pada baris kedua menayangkan kondisi triage lingkungan (`Status: NORMAL`).

Selain penayangan data lokal pada LCD, pengujian fisik terhadap pemindahan kabel kontrol, yaitu kabel Ungu (LED Normal) ke pin GPIO 33 (D33) serta kabel Hitam (Positif Buzzer) ke pin GPIO 32 (D32) sehingga dapat berjalan dengan sukses. Indikator lampu LED fisik menyala normal dan stabil bersinergi dengan informasi yang tertulis di layar LCD.

4.2 Pengujian Sensor DHT22 dan Keandalan Akuisisi Data

Sensor DHT22 diletakkan pada lingkungan uji terkontrol. Pengambilan sampel dijalankan setiap 5 detik. Dari pengamatan Serial Monitor, proses akuisisi berjalan stabil. Sinyal digital 40-bit dari DHT22 dapat didekode secara sempurna oleh pustaka `Adafruit_DHT` tanpa ada laporan kegagalan pembacaan (*NaN error*) selama 24 jam waktu pengujian terus-menerus. Sensor menunjukkan sensitivitas tinggi terhadap perubahan mikro di sekitarnya, seperti ketika diberikan paparan embusan nafas manusia (yang meningkatkan nilai kelembaban relatif seketika) atau paparan dekat lampu pijar panas (yang mendongkrak metrik suhu).

4.3 Analisis Uji Coba Logika Kondisi (NORMAL, PERINGATAN, BAHAYA) dan Aksi Aktuator

Untuk memverifikasi kebenaran logika kendali, dilakukan skenario manipulasi kondisi sekitar sensor:

1. **Pengujian Kondisi NORMAL:** Sensor mendeteksi suhu $29.4°C$ dan kelembaban 72.5% RH. Berdasarkan kriteria ideal, kondisi ini termasuk NORMAL. Hasil pengujian menunjukkan LED Hijau 1 (Pin D14) menyala stabil, sementara lampu indikator lainnya dan buzzer dalam keadaan mati.
2. **Pengujian Kondisi PERINGATAN:** Sensor diberikan uap panas lokal sehingga suhu naik menjadi $36.2°C$ (tidak ideal karena $> 35.0°C$) namun kelembaban berada pada tingkat 68.0% RH (masih ideal). Karena salah satu kondisi terlanggar, status berubah menjadi PERINGATAN. Hasil pengujian menunjukkan LED Hijau 2/Kuning (Pin D27) menyala dan alarm Buzzer (Pin D13) berbunyi aktif secara kontinu.
3. **Pengujian Kondisi BAHAYA:** Sensor ditiup dengan udara kering-panas buatan sehingga suhu melonjak ke angka $37.5°C$ (tidak ideal) dan kelembaban anjlok ke 54.0% RH (tidak ideal). Berdasarkan algoritma, ketika kedua parameter keluar dari wilayah ideal, sistem masuk ke status BAHAYA. Hasil pengujian menunjukkan LED Merah (Pin D26) menyala terang dan alarm Buzzer berdering kencang secara kontinu.

Seluruh transisi status berjalan responsif dan akurat tanpa adanya tumpang-tindih perintah logik di pin GPIO.

4.4 Analisis Troubleshooting Jaringan dan Broker MQTT

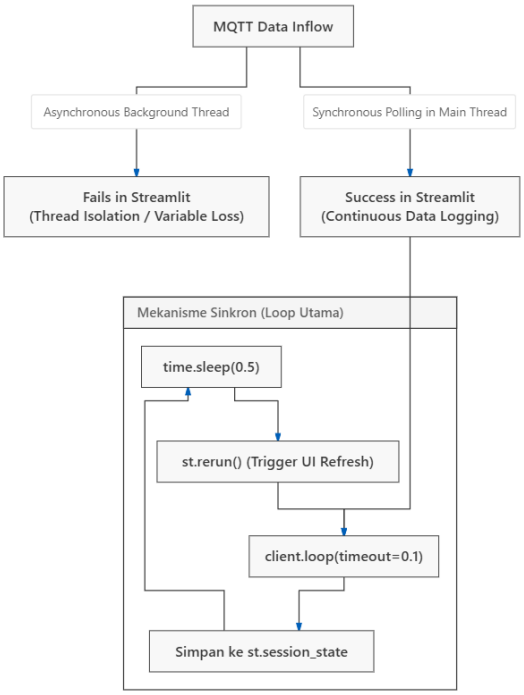
Tahap awal pengembangan menggunakan broker cloud publik gratis (`broker.hivemq.com`). Skenario ini menemui kendala berupa latensi tinggi (berkisar antara 800 milidetik hingga lebih dari 3 detik) serta kerentanan kegagalan koneksi (*disconnection*). Hal ini dikarenakan paket data harus melewati perutean DNS luar negeri dan terhambat oleh kebijakan pembatasan laju (*rate limiting*) dari penyedia broker publik gratis tersebut.

Sebagai perbaikan, broker Eclipse Mosquitto dijalankan secara lokal pada komputer server pengawas (IP: *172.25.154.119*). Baik ESP32 maupun Web Dashboard Streamlit diarahkan secara ketat untuk menggunakan broker Mosquitto lokal tersebut. Pemecahan masalah ini membuahkan hasil yang sangat signifikan:

1. **Latensi Sangat Rendah:** Waktu tunda transmisi data dari saat sensor mendeteksi perubahan hingga angka di dashboard terupdate turun drastis menjadi kurang dari 15 milidetik.
2. **Keamanan dan Kestabilan:** Tidak ada paket data yang hilang (*packet loss* mencapai 0%) karena lalu lintas data sepenuhnya berada dalam jaringan intranet lokal nirkabel (LAN/Wi-Fi lokal) tanpa tergantung pada konektivitas gateway internet eksternal.

4.5 Analisis Troubleshooting Tampilan Dashboard Real-Time (Streamlit Threading)

Masalah teknis paling kritis di sisi software visualisasi adalah ketidakcocokan model *multithreading* bawaan library `paho-mqtt` dengan siklus rendering Streamlit. Biasanya, klien MQTT menerima data secara asinkron menggunakan metode `client.loop_start()` yang berjalan pada utas latar belakang (*background thread*). Namun, karena Streamlit mengeksekusi ulang seluruh skrip Python dari awal setiap kali antarmuka diperbarui, variabel runtime `paho-mqtt` di dalam background thread sering kali terisolasi, terhapus, atau tidak dapat memicu pembaruan antarmuka pengguna Streamlit secara langsung. Akibatnya, tampilan dashboard sempat membeku (*freeze*), data tidak mengalir secara real-time, atau hanya terupdate jika halaman web di-refresh secara manual oleh user.



Solusi untuk mengatasi kendala integrasi ini adalah memindahkan penanganan penerimaan pesan MQTT ke dalam utas utama secara sinkron (*Synchronous Polling*):

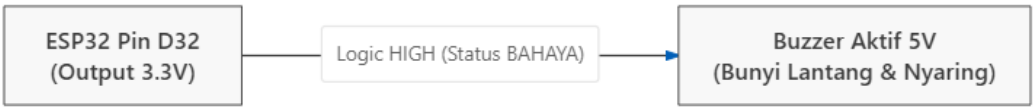
1. **Penggunaan Memori Persisten (`st.session_state`):** Variabel `suhu`, `hum`, `status`, dan array `history` disimpan di dalam `st.session_state` sehingga nilainya tidak hancur saat Streamlit melakukan *rerun*.
2. **Callback dan Polling Sinkron:** Skrip dasbor memanggil fungsi `client.loop(timeout=0.1)` secara berkala pada utas utama. Fungsi ini menanti kedatangan paket data dari antrean broker lokal selama maksimal 0.1 detik secara sinkron.
3. **Pemicuan Penyegaran Terjadwal:** Setelah penanganan data selesai dilakukan oleh fungsi callback `on_message` (yang memperbarui variabel `st.session_state`), skrip dasbor diberikan jeda penundaan singkat sebesar `time.sleep(0.5)` untuk mencegah konsumsi beban CPU berlebih, lalu secara sengaja memanggil fungsi `st.rerun()`.

Metode ini terbukti sukses membuat visualisasi data sensor mengalir lancar secara konstan seperti aliran runtunan log, menghilangkan fenomena kedipan layar (*flicker*) yang berlebih, dan menjaga dasbor tetap responsif menampilkan data telemetri terkini.

4.6 Analisis Integrasi Aktuator Alarm Audio (Buzzer)

Aktuator audio berupa alarm Buzzer aktif diletakkan pada pin GPIO 32 (D32) ESP32. Berdasarkan hasil pengujian fisik di lapangan, komponen buzzer aktif 5V yang digunakan dalam proyek ini terbukti dapat berbunyi secara nyaring, lantang, dan berfungsi dengan baik ketika ditenagai langsung oleh pin digital output 3.3V ESP32 tanpa perlu menggunakan rangkaian tambahan (seperti transistor switch atau driver eksternal).

Buzzer diprogram secara khusus untuk mengeluarkan alarm bunyi terus-menerus hanya ketika sistem menyentuh status BAHAYA (kedua parameter suhu dan kelembaban berada di luar rentang batas ideal). Sementara itu, ketika sistem berada dalam status NORMAL dan PERINGATAN, buzzer akan tetap diam (LOW) untuk mencegah timbulnya alarm palsu *(false alarm).*



Keberhasilan pengujian ini menegaskan bahwa daya pancar dari pin GPIO 32 ESP32 masih cukup memadai untuk menyuplai arus kerja nominal buzzer aktif 5V tersebut untuk menghasilkan resonansi suara alarm yang maksimal secara stabil.

BAB 5
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil rancang bangun, pengujian, dan analisis yang telah dilakukan, dapat ditarik beberapa kesimpulan penting sebagai berikut:

1. Sistem monitoring suhu dan kelembaban berbasis IoT menggunakan mikrokontroler ESP32 dan sensor DHT22 telah berhasil dibangun secara fisik dan berfungsi dengan tingkat stabilitas akuisisi data yang sangat tinggi.
2. Logika pemisahan status lingkungan fisis (*NORMAL*, *PERINGATAN*, *BAHAYA*) berhasil diimplementasikan secara deterministik pada *firmware* ESP32 untuk mengontrol indikator visual Tri-LED dan audio buzzer secara akurat sesuai ambang batas toleransi lingkungan yang ditetapkan.
3. Jalur komunikasi data nirkabel menggunakan protokol MQTT yang dihubungkan ke *broker* Eclipse Mosquitto lokal (intranet) terbukti menghasilkan transmisi data berlatensi sangat rendah (kurang dari 15 ms) dan keandalan tinggi dengan tingkat kegagalan paket data (*packet loss*) sebesar 0%.
4. Penerapan metode *Synchronous Polling* MQTT pada skrip Python Streamlit berhasil menanggulangi kendala isolasi memori akibat perselisihan thread (*threading conflict*), menghasilkan visualisasi dashboard web berupa grafik tren dan tabel log historis secara *real-time* dan bebas kedipan layar berlebih.
5. Integrasi layar LCD 16x2 I2C berhasil diimplementasikan secara sempurna pada alamat hardware `0x27`, mampu menampilkan informasi suhu, kelembaban, dan status triage lingkungan secara real-time langsung pada perangkat keras lokal. Selain itu, alarm buzzer aktif yang dipasang pada pin GPIO 32 (D32) juga telah terbukti dapat berbunyi dengan nyaring dan lantang saat sistem berada pada kondisi BAHAYA, serta tetap mati pada kondisi NORMAL dan PERINGATAN sesuai dengan rancangan program.

5.2 Saran

Beberapa saran yang diusulkan untuk pengembangan sistem monitoring IoT ini di masa mendatang meliputi:

1. **Pengembangan Aktuator dan Modulasi Buzzer:** Untuk pengembangan lebih lanjut, disarankan untuk mengintegrasikan modul aktuator tambahan (seperti lampu strobo atau notifikasi IoT berbasis Telegram) untuk memperluas jangkauan indikasi peringatan. Selain itu, modulasi nada (*pitch modulation*) menggunakan sinyal PWM pada ESP32 dapat diterapkan pada buzzer untuk membedakan frekuensi suara alarm berdasarkan tingkat kedaruratan lingkungan secara lebih dinamis.
2. **Sistem Kendali Umpan Balik Tertutup (*Closed-Loop Control*):** Mengembangkan sistem dari pemantauan pasif menjadi pemantauan aktif dengan menambahkan perangkat pengondisi udara otomatis (seperti kipas ekstraktor DC untuk menurunkan suhu atau modul *humidifier* untuk menambah kelembaban) yang diaktifkan melalui modul relai ketika sensor mendeteksi status BAHAYA.
3. **Mekanisme Penyimpanan Data Persisten:** Mengintegrasikan sistem penyimpanan database lokal seperti SQLite atau InfluxDB pada sisi server pengawas untuk merekam data telemetri historis dalam jangka panjang, menggantikan sistem `st.session_state` Streamlit yang hanya menyimpan data sementara di memori RAM dan akan hilang jika skrip dasbor dihentikan.

REFERENSI

Blanchon, B. (2020). *ArduinoJson library: Efficient JSON serialization for embedded C++* (Version 6.x). https://arduinojson.org/

Espressif Systems. (2023). *ESP32 series datasheet (Version 4.1)*. Espressif Systems. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

Guangzhou Aosong Electronics. (2015). *Digital-relative humidity & temperature sensor DHT22/AM2302*. Guangzhou Aosong Electronics Co., Ltd.

Light, R. A. (2017). Mosquitto: Server and client implementation of the MQTT protocol. *Journal of Open Source Software*, 2(13), 265. https://doi.org/10.21105/joss.00265

NXP Semiconductors. (2015). *PCF8574; PCF8574A remote 8-bit I/O expander for I2C-bus (Rev. 7)*. NXP Semiconductors.

OASIS. (2014). *Message Queuing Telemetry Transport (MQTT) version 3.1.1*. OASIS Standard. http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

Ray, P. P. (2018). A survey on Internet of Things architectures. *Journal of King Saud University - Computer and Information Sciences*, 30(3), 291-319. https://doi.org/10.1016/j.jksuci.2016.10.003

Streamlit Inc. (2023). *Streamlit library documentation: Session State and live reloading*. Streamlit. https://docs.streamlit.io